COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	15/09/2026	**DATE SUBMITTED:**	15/09/2026
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO.8.1 **Sorting Algorithms Pt2**

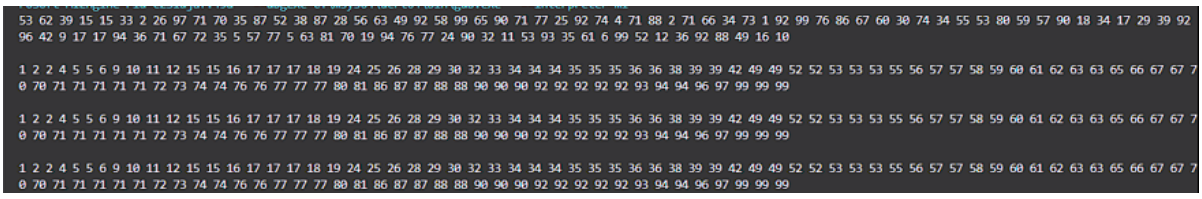
1. PROCEDURE OUTPUT

Screenshot

```
template <typename T>
void generateRandomArray(T arr[], int size) {
    srand(static_cast<unsigned int>(time(nullptr)));
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 100;
    }
}

template <typename T>
void displayArray(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
}
```

OUTPUT



Observation

We're starting off with a completely random list of 100 numbers ranging from 0 to 99. To keep the comparison totally fair, every single sorting algorithm gets its own exact copy of this original, unsorted batch to work with.

Table 8-1. Array of Values for Sort Algorithm Testing

Code

```
template <typename T>
void shellSort(T arr[], int arrSize) {
    int gap, i, j;
    T temp;

    gap = arrSize / 2;
    while (gap > 0) {
        for (i = gap; i < arrSize; i++) {
            temp = arr[i];
            j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap]; // shift up by the gap
                j -= gap;
            }
            arr[j] = temp; // drop it into place
        }
        gap /= 2; // shrink the gap each pass (medyo iba process neto)
    }
}
```

Output

```
53 62 39 15 15 33 2 26 97 71 70 35 87 52 38 87 28 56 63 49 92 58 99 65 90 71 77 25 92 74 4 71 88 2 71 66 34 73 1 92 99 76 86 67 60 30 74 34 55 53 80 59 57 90 18 34 17
96 42 9 17 17 94 36 71 67 72 35 5 57 77 5 63 81 70 19 94 76 77 24 90 32 11 53 93 35 61 6 99 52 12 36 92 88 49 16 10

1 2 2 4 5 5 6 9 10 11 12 15 15 16 17 17 17 18 19 24 25 26 28 29 30 32 33 34 34 34 35 35 35 36 36 38 39 39 42 49 49 52 52 53 53 53 55 56 57 57 58 59 60 61 62 63 63 65 6
0 70 71 71 71 71 71 72 73 74 74 76 76 77 77 77 80 81 86 87 87 88 88 90 90 90 92 92 92 92 92 93 94 94 96 97 99 99 99

1 2 2 4 5 5 6 9 10 11 12 15 15 16 17 17 17 18 19 24 25 26 28 29 30 32 33 34 34 34 35 35 35 36 36 38 39 39 42 49 49 52 52 53 53 53 55 56 57 57 58 59 60 61 62 63 63 65 6
0 70 71 71 71 71 71 72 73 74 74 76 76 77 77 77 80 81 86 87 87 88 88 90 90 90 92 92 92 92 92 93 94 94 96 97 99 99 99

1 2 2 4 5 5 6 9 10 11 12 15 15 16 17 17 17 18 19 24 25 26 28 29 30 32 33 34 34 34 35 35 35 36 36 38 39 39 42 49 49 52 52 53 53 53 55 56 57 57 58 59 60 61 62 63 63 65 6
0 70 71 71 71 71 71 72 73 74 74 76 76 77 77 77 80 81 86 87 87 88 88 90 90 90 92 92 92 92 92 93 94 94 96 97 99 99 99
```

Observations

Shell sort got the array in order by gradually shrinking the gap size, starting at half the total length and stepping all the way down to 1. Sorting items that were far apart first saved the algorithm from all the slow, one-by-one shifting that standard insertion sort usually gets stuck doing.

Table 8-2. Shell Sort Technique

Code

```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left >= right) return; // base case, 0 or 1 element left

    int mid = (left + right) / 2;
    mergeSort(arr, left, mid);      // sort the left half
    mergeSort(arr, mid + 1, right); // sort the right half
    merge(arr, left, mid, right);   // merge them back together
}
```

```cpp
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int leftSize = mid - left + 1;
    int rightSize = right - mid;
    int i, j, k;

    T* leftArr = new T[leftSize];
    T* rightArr = new T[rightSize];

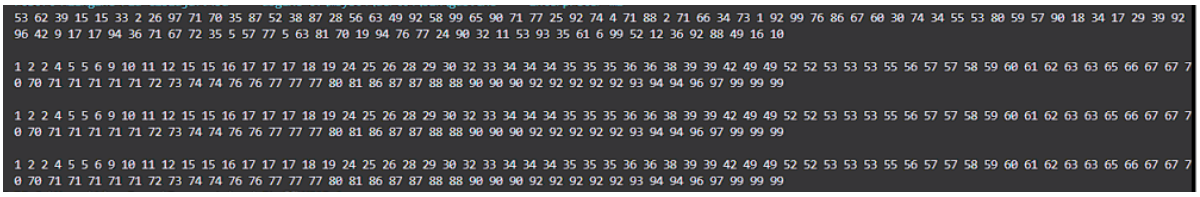
    for (i = 0; i < leftSize; i++) leftArr[i] = arr[left + i];
    for (j = 0; j < rightSize; j++) rightArr[j] = arr[mid + 1 + j];

    i = 0, j = 0, k = left;
    while (i < leftSize && j < rightSize) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i]; // take from the left half
            i++;
        } else {
            arr[k] = rightArr[j]; // take from the right half
            j++;
        }
        k++;
    }

    while (i < leftSize) { arr[k] = leftArr[i]; i++; k++; }  // leftover left elements
    while (j < rightSize) { arr[k] = rightArr[j]; j++; k++; } // leftover right elements

    delete[] leftArr;
    delete[] rightArr;
}
```

Output



Observations

Merge sort handled the array by continuously cutting it in half until every element was on its own. From there, it just merged those tiny pieces back together in order, leaving us with a fully sorted list at the end.

Table 8-3. Merge Sort Algorithm

Code

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high]; // last element chosen as pivot
    int i = low - 1;
    int j;

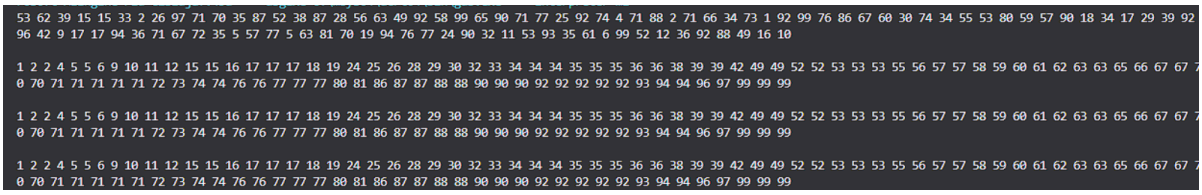
    for (j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]); // pivot lands in its final spot
    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low >= high) return; // 0 or 1 element, nothing to do

    int pivotPos = partition(arr, low, high);
    quickSort(arr, low, pivotPos - 1);  // left sub list
    quickSort(arr, pivotPos + 1, high); // right sub list
}

#endif
```

Output



Observations

Quick sort got the array in order by constantly picking the last element as a pivot to split things up. It shifted all the smaller numbers to the left and larger ones to the right, then just ran that same process on both halves until everything fell into place.

Table 8-4. Quick Sort Algorithm

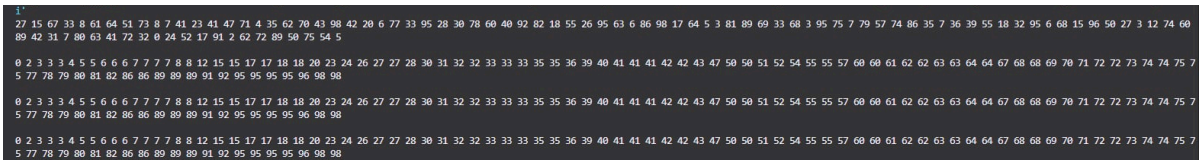
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

```
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low >= high) return; // 0 or 1 element, nothing to d

    int pivotPos = partition(arr, low, high);
    quickSort(arr, low, pivotPos - 1);  // left sub list
    quickSort(arr, pivotPos + 1, high); // right sub list
}
```

OUTPUT



Problem 1

Since each `partition()` step just isolates contiguous chunks of the original array, you aren't forced to stick strictly to recursive Quick sort calls for those sub-lists. Once the main array is split into smaller left and right sections around the pivot, you could theoretically drop in almost any other sorting algorithm like Insertion sort or Heap sort to finish sorting those specific index ranges directly in place.

Problem 2
For a smaller array of 15 elements, algorithms like Quick sort or Insertion sort run faster because Merge sort wastes time setting up extra temporary arrays. However, as the dataset gets much bigger, Quick sort really takes off and outperforms the rest due to its efficient average-case runtime. Both Merge and Quick sort hit that optimal $O(N \log N)$ complexity thanks to their divide-and-conquer strategy, which splits the array in half across $\log N$ levels of recursion while doing $O(N)$ work at each step.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.